Data Indexing in Peer-to-Peer DHT Networks

L. Garcés-Erice, P.A. Felber, E.W. Biersack, G. Urvoy-Keller
Institut EURECOM, 06904 Sophia Antipolis, France
{garces|felber|erbi|urvoy}@eurecom.fr

K.W. Ross
Polytechnic University, Brooklin, NY 11201 , USA
ross@poly.edu

Abstract

Peer-to-peer distributed hash table (DHT) systems make it simple to discover specific data when their complete identifiers—or keys—are known in advance. In practice, however, users looking up resources stored in peer-to-peer systems often have only partial information for identifying these resources. In this paper, we describe techniques for indexing data stored in peer-to-peer DHT networks, and discovering the resources that match a given user query. Our system creates multiple indexes, organized hierarchically, which permit users to locate data even using scarce information, although at the price of a higher lookup cost. The data itself is stored on only one (or few) of the nodes. Experimental evaluation demonstrates the effectiveness of our indexing techniques on a distributed peer-to-peer bibliographic database with realistic user query workloads.

1. Introduction

Peer-to-peer (P2P) systems make it possible to harness the computing power and resources of large populations of networked computers in a cost-effective manner. In this paper, we focus on P2P systems where data items are spread across distributed peer computers (nodes) and the location of each item is determined in a decentralized manner using a distributed hash table (DHT), such as Chord [1], CAN [2], Pastry [3], or Tapestry [4].

A major limitation of P2P DHT systems is that they only support exact-match lookups: one needs to know the exact key (identifier) of a data item to locate the node(s) responsible for storing that item. In practice, however, P2P users often have only partial information for identifying these items and tend to submit broad queries (e.g., all the articles written by "John Smith").

In this paper, we propose to augment P2P DHT systems with mechanisms for locating data using incomplete information. Note that we do not aim at answering complex database-like queries, but rather at providing practical techniques for searching data in a DHT. Our mechanisms rely on indexes, stored and distributed across the nodes of the network, that maintain useful information about user queries. Given a broad query, a user can obtain additional information about the data items that match his original query; the DHT can be recursively queried until the user finds the desired data items. Indexes can be organized hierarchically to reduce space and bandwidth requirements, and to facilitate interactive searches. They integrate an adaptive distributed cache to speed up accesses to popular content.

Our indexing techniques can be layered on top of an arbitrary P2P DHT infrastructure, and thus benefit from any advanced features implemented in the DHT (e.g., replication, load-balancing). We have conducted a comprehensive evaluation that demonstrates their effectiveness in realistic settings. In particular, we have observed that our techniques are scalable, adapt well to user search patterns, and have reasonably-small space requirements. Look-up times depend on the "precision" of the initial query: broad queries incur higher lookup times than specific queries.

The organization of this paper is as follows: In Section 2, we discuss related work, and we introduce the system model in Section 3. We describe our distributed indexing techniques in Section 4 and we evaluate them in Section 5. Finally, Section 6 concludes the paper.

Figure 1. Sample file descriptors.

2. Related Work

Of the various related project, INS/Twine [5] is most similar to our work. INS/Twine is an architecture for intentional (i.e., based on what we are looking for, not where it is located [6]) resource discovery, which allows to easily locate services and devices in large scale environments, using intentional descriptions. INS/Twine works on top of a DHT, such as Chord [1], by setting up a number of resolvers, which collaborate to distribute resource information and to resolve simple queries. Given a semi-structured resource description, INS/Twine extracts prefix subsequences of attributes

Proceedings of the 24th International Conference on Distributed Computing Systems (ICDCS'04)
1063-6927/04 $20.00 © 2004 **IEEE**

and values, called "strands". INS/Twine then computes the hash values for each of these strands, which constitutes numeric keys used to map resources to resolvers. The resource and device information are stored redundantly on *all* peer resolvers that correspond to the numeric keys. When looking up some resource, INS/Twine sends the query to the resolver node identified by one of the longest strands; the query is further processed by the resolver, which returns the matching resource descriptions.

Unlike Twine, we do not replicate data at multiple locations; we rather provide a key-to-key service, or more precisely a query-to-query service. We do not introduce dedicated resolvers in our architecture; we only require the underlying distributed data storage system to allow for the registration of multiple entries using the same key. As we allow index keys to be tree-structured or non-prefix sub-keys, data can be looked up using more expressive and selective queries. Index entries are organized hierarchically to improve scalability.

In [7], the authors discuss techniques for performing complex queries in DHT-based P2P networks, using traditional relational database operators (selection, projection, join, grouping and aggregation, and sorting) and elaborate text retrieval techniques (like splitting a query string and using each piece to create a key matching the query).

In [8], the authors develop a P2P data sharing architecture for computing approximate answers for complex queries by finding data ranges that are similar to the user query. Relevant data is located using "locality sensitive hashing" techniques. In [9], the same authors extend the CAN [2] system to support the basic range operation on data shared in the form of database relations.

More recently, the authors of [10] combine the flexibility of searches in unstructured P2P networks (like Gnutella [11]) with peer grouping to form associative overlays. Peers are organized in groups with common interests. Groups are located by routing queries, and searches are performed within groups using more flexible algorithms. In [12], the authors question the feasibility of an Internet wide search engine based on P2P, but their conclusion do not apply to our techniques, which are targeted at smaller scale systems with well-specified content.

3. System Model and Definitions

The distributed indexes are layered on top of a P2P DHT infrastructure—or substrate—and use various other technologies to describe content and represent user queries. We now describe the specific requirements of our indexing techniques.

DHT Storage. A DHT system maps keys to nodes in a peer-to-peer infrastructure. Any node can use the DHT substrate to determine the current live node that is responsible for a given key. In this paper, we assume an underlying DHT-based P2P data storage system, in which each data item is mapped to one or several peer nodes. Example of such systems are Chord/DHash/CFS [13] and Pastry/PAST [14].

Throughout the paper, we will use the example of a bibliographic database system that stores scientific articles. Files are identified by *descriptors*, which are textual, human-readable descriptions of the file's content. Let $h(descriptor)$ be a hash function that maps identifiers to a large set of numeric keys. The peer node responsible for storing a file f is determined by transforming the file's descriptor d into a numeric key $k = h(d)$. This numeric key is used by the DHT substrate to determine the node responsible for f.

Data Descriptors and Queries. We assume that descriptors are semi-structured XML data [15], as used by many publicly-accessible databases (e.g., DBLP [16]). Examples of descriptors for bibliographic data are given in Figure 1. These descriptors have fields useful for searching files (author, title), as well as fields useful for an administrator of the database (size).

To search for data stored in the peer-to-peer substrate, we need to specify broad queries that can match multiple file descriptors. For this purpose, we use a subset of the XPath XML addressing language [17], which offers a good compromise between expressiveness and simplicity. XPath treats XML documents as a tree of nodes and offers an expressive way to specify and select parts of this tree. An XPath expression contains one or more *location steps*, separated by slashes (/). In its more basic form, a location step designates an element name followed by zero or more predicates specified between brackets. Predicates are generally specified as constraints on the presence of structural elements, or on the values of XML documents using basic comparison operators. XPath also allows the use of wildcard (*) and ancestor/descendant (//) operators, which respectively match exactly one and an arbitrarily long sequence of element names. An XML document (i.e., a file descriptor) *matches* an XPath expression when the evaluation of the expression on the document yields a non-null object.

```
q1 = /article[author[first/John][last/Smith]] ···
     [title/TCP][conf/SIGCOMM][year/1989][size/315635]
q2 = /article[author[first/John][last/Smith]][conf/INFOCOM]
q3 = /article/author[first/John][last/Smith]
q4 = /article/title/TCP
q5 = /article/conf/INFOCOM
q6 = /article/author/last/Smith
```

Figure 2. Sample file queries.

For a given descriptor d, we can easily construct an XPath expression (or query) q that tests the presence of all the elements and values in d.[1] We call this expression the *most specific query* for d or, by extension,

1 In fact, we can create several equivalent XPath expressions for the same query. We assume that equivalent expressions are transformed into a unique normalized format.

the *most specific descriptor*. Conversely, given q, one can easily construct d, compute $k = h(d)$, and find the file. For instance, query q_1 in Figure 2 is the most specific query for descriptor d_1 in Figure 1.

Given two queries q and q', we say that q' *covers* q (or q is covered by q'), denoted by $q' \sqsupseteq q$, if any descriptor d that matches q also matches q'. Abusing the notation, we often use d instead of q when q is the most specific query for d and we consider them as equivalent ($q \equiv d$); in particular, we say that q' covers d when $q' \sqsupseteq q$ and q is the most specific query for d. It should be noted that the covering relation introduces a partial ordering on the queries.

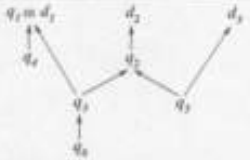

Figure 3. Partial ordering tree for the queries of Figure 2 (trivial relations omitted).

The partial ordering graph for queries in Figure 2 is shown in Figure 3, where $q_i \rightarrow q_j$ is read $q_i \sqsupseteq q_j$ (more specific queries are represented above less specific queries).

4. Indexing

When the most specific query for the descriptor d of a file f is known, finding the location of f is straightforward using the key-to-node (and hence key-to-data) underlying DHT lookup service. The goal of our architecture is to also offer access to f using less specific queries that cover d.

The principle underlying our technique is to generate multiple keys for a given descriptor, and to store these keys in indexes maintained by the DHT infrastructure. Indexes do not contain key-to-data mappings; instead, they provide a key-to-key service, or more precisely a query-to-query service. For a given query q, the index service returns a (possibly empty) list of more specific queries, covered by q. If q is the most specific query of a file, then the DHT storage system returns the file (or indicates the node responsible for that file). By iteratively querying the index service, a user can traverse upward the partial order graph of the queries (see Figure 3) and discover all the indexed files that match his broad query.

In order to manage indexes, the underlying DHT storage system must be slightly extended. Each node should maintain an index, which essentially consists of query-to-query mappings. The "$insert(q, q_i)$" function, with $q \sqsupseteq q_i$, adds a mapping $(q; q_i)$ to the index of the node responsible for key q. The "$lookup(q)$" function, with q not being the most specific query of a file, returns a list of all the queries q_i such that there is a mapping $(q; q_i)$ in the index of the node responsible for key q.

Roughly speaking, we store files and construct indexes as follows: Given a file f and its descriptor d, with a corresponding most specific query q, we first store f at the node responsible for the key $k = h(q)$. We generate a set of queries $Q = \{q_1, q_2, \ldots, q_l\}$ likely to be asked by users (to be discussed shortly), and such that each $q_i \sqsupseteq q$. We then compute the numeric key $k_i = h(q_i)$ for each of the queries, and we store a mapping $(q_i; q)$ in the index of the node responsible for k_i in the DHT. Optionally, we iterate the process shown for q with every q_i and recursively until all the desired index entries have been created.

Figure 4. Sample indexing scheme for a bibliographic database.

Example. To best illustrate the principle of our indexing techniques, consider a P2P bibliographic database that stores the three files associated to the descriptors of Figure 1. We want to be able to lookup publications using various combinations of the author's name, the title, the conference, and the publication year. A possible hierarchical indexing scheme is shown in Figure 4. Each box corresponds to a distributed index, and indexing keys are indicated inside the boxes. The index at the origin of an arrow stores mapping between its indexing key and the indexing key of the target. For instance, the *Last name* index stores the full names of all authors that have a given last name; the *Author* index maintains information about all articles published by a given author; the *Article* index stores the descriptors (MSDs) of all publications with a matching title and author name.

After applying this indexing scheme to the three files of the bibliographic database, we obtain the distributed indexes shown in Figure 5 (Top). The top-level *Publication* index corresponds to the raw entries stored in the underlying DHT-based storage system: complete key provide direct access to the associated files. The other indexes hold query-to-query mappings that enable the user to iteratively search the database and locate the desired files. Each entry of an index is potentially stored on a different node in the DHT substrate, as illustrated for the *Proceedings* index. One can observe that some index entries associate a query to multiple queries (e.g., in the *Author* index).

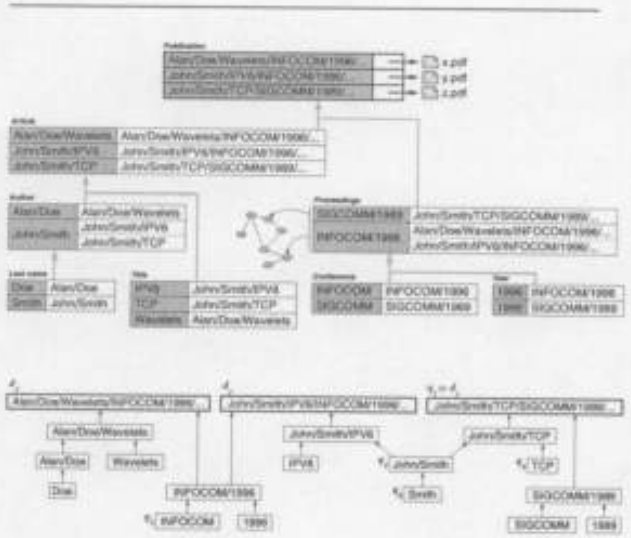

Figure 5. (Top) Sample distributed indexes for the three documents of Figure 1 and the indexing scheme of Figure 4. (Bottom) Corresponding query mappings with identifiers from Figures 1 and 2 (query syntax has been simplified).

Figure 5 (Bottom) details the individual query mappings stored in the indexes of Figure 5 (Top). Each arrow corresponds to a query-to-query mapping, e.g., $(q_6; q_3)$. The files corresponding to descriptors d_1, d_2, and d_3 can be located by following any valid path in the partial order tree. For instance, given q_6, a user will first obtain q_3; the user will query the system again using q_3 and obtain two new queries that link to d_1 and d_2; the user can finally retrieve the two files matching its query using d_1 and d_2.

Lookups. We can now describe the lookup process more formally. When looking up a file f using a query q, a user first contacts the node n responsible for $h(q)$. That node may return f if q is the most specific query for f, or a list of queries $\{q_1, q_2, \ldots, q_n\}$ such that the mappings $(q; q_i)$, with $q \sqsupseteq q_i$, are stored at n. The user can then choose one or several of the q_i and repeat this process recursively until the desired files have been found. The user effectively follows an "index path" that leads from q to f.

Lookups may require several iterations when the most specific query for a given file is not known. Higher index hierarchy usually necessitate more iterations to locate a file, but are also generally more space-efficient, as each index factorizes in a compact manner the queries of its child indexes. In particular, the size of the lists (result sets) returned by the index service may be prohibitively long when using a flat indexing scheme (consider, for example, the list of all articles written by the persons whose last name is "Smith"). There is therefore a trade-off between space requirements, size of result sets, and lookup time. Notice that the coarse-level indexes are shared among many data items.

The lookup process can be interactive, i.e., the user directs the search and restricts its query at each step, or automated, i.e., the system recursively explores the indexes and return all the file descriptors that match the original query.

When a user wants to look up a file f using a query q, it may happen that q is not present in any index, although f does exist in the peer-to-peer system and q is a valid query for f. In that case, it is still possible to locate f by (automatically) looking for a query q_i such that $q_i \sqsupseteq q$ and q_i is on some index path that leads to f.

For instance, given the distributed indexes of Figure 5 it appears that query q_2 in Figure 2 is not present in any index (q_2 = `/article[author[first/John][last/Smith]][conf/INFOCOM]`). We can however find q_3, such that $q_3 \sqsupseteq q_2$ and there exists an index path from q_3 to d_1. Therefore, the file associated to d_1 can be located using this generalization/specialization approach, although at the price of a higher lookup cost. We believe that it is natural for lookups performed with less information to require more effort. Also, the different paths to data spread the load among different nodes.

Building and Maintaining Indexes. When a file is inserted in the system for the first time, it has to be indexed. The choice of the queries under which a file is indexed is arbitrary, but the covering relation must hold. Arbitrary links (or aliases) to a file cannot be inserted in the system. This makes it harder for a user to masquerade malicious or offensive data as another existing file. As files are discovered using the index entries, a file is more likely to be located rapidly if it is indexed "enough" times, under "likely" names. The quantity and likelihood of index queries are hard to quantify and are often application-dependent.

Note that the length of the index paths that lead to a given file is arbitrary, although it directly affects the lookup time. Less popular content may be indexed using a deeper index hierarchy, to reduce space and bandwidth requirements, and to facilitate interactive searches. In contrast, a very popular file can be linked to deep in the hierarchy to short-circuit some indexes and speed up lookups. For instance, given the indexes of Figure 5, one can add the $(q_6; d_1)$ index entry to speed up searches for the popular file described by d_1.

Note also that more generic queries can be obtained from more specific queries by removing only portions of element names (i.e., using substring matching). For instance, one can create an index with all the files of an author that start with the letter "A", the letter "B", etc. One can also envision to use techniques similar to those discussed in [7] for substring matching. In general, determining good decompositions for indexing each given descriptor type (e.g., articles, music files, movies, books, etc.) requires human input.

In a system model where files are injected in the system, but are never deleted (write-once semantics), index entries never need to be updated. In a read/write system, when a file is deleted we have to find all the indexes that refer to the descriptor of that file and delete the associated mappings. Locating the index entries can be achieved straightforwardly by the same process used to generate them. When deleting the last mapping for a given key, we can recursively delete the references to that key to clean up the indexes.

Index entries can also be created dynamically to adapt to the users query patterns. For instance, a user who tries to locate a file f using a non-indexed query, and eventually finds it using the query generalization/specialization approach discussed above, can add an index entry to facilitate subsequent lookups from other users.

More interestingly, one can easily build an adaptive cache in the DHT to speed up accesses to popular files. Assume that each node allocates a limited number of index entries for caching purposes. After a successful lookup, a peer can create "shortcuts" entries (i.e., direct mappings between generic queries and the descriptor of the target file) in the caches of the indexes traversed during the lookup process. Another user looking for the same file via the same path will be able to "jump" directly to the file by following the shortcuts stored in the caches. With a least-recently used (LRU) cache replacement policy, we guarantee that the most popular files are well represented in the caches. The caching mechanism therefore adapts automatically to the query patterns and file popularity. Finally, note that indexes are treated like regular data, so they can benefit from the mechanisms implemented by the DHT for increasing availability and scalability (load-balancing, replication).

5. Evaluation of Data Indexing on P2P Networks

The indexing mechanism lies in the protocol stack on top of a P2P lookup and storage layer. Thus, one aspect of indexing performance deals with the optimization of resources offered by those lower layers. As our indexing techniques do not depend on a specific lookup and storage layer, we do not explicitly study the performance of the P2P substrate. The index protocol layer also interacts with the end user, who expects to find data of interest using partial information. From the user point of view, it is clearly desirable to find the desired data in a minimal number of interactions with the system, and the information returned by the system should be as concise and relevant as possible. From the system point of view, the search process should be simple, the amount of network traffic should be minimized, and the storage space dedicated to the indexing metadata should remain within reasonable limits.

Distributed Bibliographic Database. To study the behavior of a P2P indexed network, we model a bibliographic database distributed among interconnected hosts. The bibliographic database contains articles published in journals and conference proceedings. The underlying P2P lookup and storage system, and the physical characteristics of the network, are not important: we simply assume that the underlying DHT is able to find a node n responsible for a given key k, where n stores (or knows the location of) the data associated with key k.

In order to build the bibliographic database, we used the publicly-available DBLP [16] archive, which consists of an XML-formatted list of publications. The DBLP archive, as of January 21^{st}, 2003, contains more than 346,000 entries: articles, theses, proceedings, etc. For our experiments we used the 115,879 article entries in the archive to build the descriptors (MSDs) of the corresponding articles. The archive being in XML format, entries are pretty similar to those in Figure 1 (actual field names differ).

Building Indexes. From the MSD, which actually links to the real data, we build chains of queries, i.e., sequences of queries where each query covers ($\sqsupseteq$) the next one. The last member of each chain is obviously the MSD, which is covered by all the previous ones. Each chain corresponds to a path from a leaf to a root in Figure 3.

To create indexes that correspond to queries that users are likely to ask, we have observed the query logs of two other bibliographic database sites: BibFinder [18] and NetBib [19] (the DBLP service does not store and share query logs). BibFinder offers an interface where a user can search for papers using fields like: the author name, title, conference or journal, and year of publication (exact, or published before/after a given year). NetBib offers a similar interface.

The log from BibFinder's site contains 9,108 queries. As shown in Figure 7 (Left), most of them use only the "author" field (57%), followed by those using only the "title" field (20%), and those with a given publication date. The NetBib trace represents 5,924 different queries. Similarly to the BibFinder traces, there are three main kinds of queries (summing up to more than 95% of the total): queries for a given author, topic (title), and date of publication.

Based on this information, we have simulated and compared the following three indexing schemes, shown in Figure 6.

Simple: A query for an author or a title returns a set of author and title pairs. Each of them points to a MSD. A query for a conference or a year returns a set of conference-year pairs, pointing to a MSD.

Flat: All possible queries in the *simple* scheme point directly to the MSD, so that the index query length is always 2.

Complex: Some queries in the *simple* scheme are split